System Design

Introduction

Vincent Tran

What is system design?

Also called architecture design. It covers:

- Functional and non-functional requirements
- Capacity estimation
- Relational and NoSQL databases
- Vertical scaling, horizontal scaling, sharding
- Load balancing
- Primary–secondary replication
- Caches and CDNs
- Stateless and stateful servers
- Datacenter georouting
- Message queues, publish–subscribe
- Performance metrics, monitoring, logging
- Build, test, configure, and deploy automation
- Finding single points of failure
- API rate limiting
- Service level agreements

Resources for system design interviews

1. Sukumar Ghosh, *Distributed Systems: An Algorithmic Approach*
2. The System Design Primer:
   `https://github.com/donnemartin/system-design-primer`
3. Roberto Vitillo, *Understanding Distributed Systems*, 2nd ed.

How we'll learn

Lecture	Topics
1. Introduction	The scenario, product ideas, the MVP, high-level design
2. High-level and component design	The model design, the thirteen component questions
3. Component implementation	Approaching a new codebase
4. Code review and analysis	Reading code, the cost of code

Welcome to Sudsley!

Welcome to Sudsley!

Today, you're joining the founding engineering team of a disruptive new startup.

The problems we're solving

1. **Wasted time.** Customers travel to and wait at the laundromat, unsure when their machine will be done.

The problems we're solving

1. **Wasted time.** Customers travel to and wait at the laundromat, unsure when their machine will be done.
2. **Inconvenience.** Machines are occupied, forcing customers to wait or come back later.

The problems we're solving

1. **Wasted time.** Customers travel to and wait at the laundromat, unsure when their machine will be done.
2. **Inconvenience.** Machines are occupied, forcing customers to wait or come back later.
3. **Forgotten clothes.** Clothes are often left in machines long after the cycle finishes.

Our secret weapon: the smart laundry machine

At the heart of Sudsley is our proprietary, next-generation washer and dryer. Some very smart engineers spent years developing it.

Every machine is internet-enabled, with an integrated API for complete remote control and monitoring:

- `startCycle()`
- `stopCycle()`
- `getStatus()`: available, running, finished, time remaining, needs maintenance

Your job

1. Explore product options and suggest ideas.
2. Design the product.
3. Implement the core components.

Product ideas

Product ideas

What software should we build, and why?

Discuss in groups.

Great ideas! But they don't matter that much

The CEO and product managers (with your completely relevant input) have settled on an idea: UberEats, Postmates, or DoorDash, but for laundry.

How they want it to work:

1. A customer puts their laundry out on their front porch.
2. They open an app to request a launderer.
3. The launderer picks up the clothes and takes them to a Sudsley location.
4. The launderer puts the laundry in a machine.
5. The app keeps the customer up to date on the status of their clothes.
6. The launderer delivers the clothes back once they're finished.
7. The launderer gets a portion of the job revenue.

Minimum viable product

Define the MVP requirements

The minimum viable product is an early version of a product with just enough features to be functional.

Guidelines for an MVP:

- Identify the must-have features.
- Avoid unnecessary work.
- Expect feedback from early users. This is not the forever-system.

In groups: define a rough MVP for the alpha version of Sudsley.

Must-haves

- Assume about 10k initial customers
- Customers can request a laundry job
- Launderers can accept a laundry job
- Launderers know which machine to put the clothes in
- Customers know where their clothes are geographically
- Customers know the status of their laundry job
- Customers pay through the app
- Launderers get paid through the app
- Customers can rate their laundry job
- Basic statistics and telemetry

High-level design

Define the high-level design

High-level (or macro-level) design is the first formal step of system design. It should be easy to follow.

- Identifies the necessary components (microservices, external services, clients) without detailing their implementation
- Aligns everyone on the project's goals and helps distribute responsibilities

May also include:

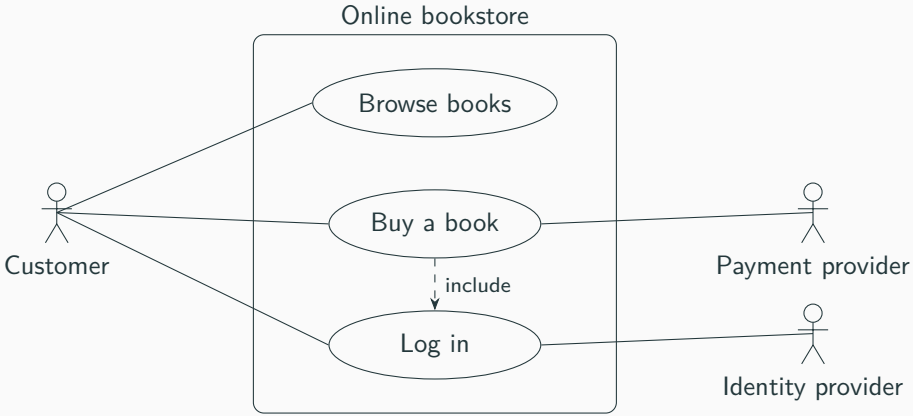
- UI designs and data flow diagrams
- The technology stack
- The deployment procedure

Design tools

Many high-level designs are drawn as UML diagrams, though this is not strictly required.

- draw.io: `https://app.diagrams.net/`
- Lucidchart: `https://www.lucidchart.com/`
- Mermaid: `https://mermaid.js.org/`

Example: a use-case diagram



Homework 1: high-level design (collaborative)

1. Based on the product and MVP requirements from class, write a brief description of how you think the system should look and operate. No more than half a page.
2. Create a high-level design for the Sudsley system.
   - A business or system use-case diagram is a good start.
   - You don't need to follow UML strictly. Make it readable and intuitive.

Be flexible! No design is correct or incorrect.

Full handout: `assignments/hw1-high-level-design/HOMEWORK.md`